User Information Needs and Ranking Effectiveness

IFN647 – Week 6

Professor Yuefeng Li
School of Computer Science
Queensland University of Technology

0

Learning Objectives

Topic ID	Key topics	Mapping to slides
1	Information filtering and Information Needs	2-5
2	Query-Based Methods for Capturing Information Needs	6-14
3	Relevance feedback using machine learning	15-18
4	Ranking Effectiveness	19-24
5	Interpolation	25-27
6	Discounted Cumulative Gain	28-33
	References	34

1

1. Information Filtering and Information Needs

- Information Filtering (IF) is a name used to describe a variety of processes involving the delivery of information to people (users).
- IF systems have a similar function to IR systems.
- Most IR systems support the short-term needs of a diverse set of users. IF systems, however, are commonly personalized to support long-term, relatively stable, or periodic goals or desires of a particular user or a group of users.
  - For example, document filtering, where the user's information need stays the same, but the document collection itself is dynamic, with new documents arriving periodically.
- The main distinction between IR and IF is that IR systems use "query", and IF systems use "user information needs (or user profiles)":
  - Typically, information needs represent long-term information interests or needs (e.g., keeping up-to-date on a topic) that may change slowly over time as conditions, goals, and knowledge change.

- It is very difficult to capture long-term information needs and represent them in a machine-readable format.
- The easy way is to use a query, but a query doesn't clearly describe what the user wants.
  - For example, for a given query {*Java*, *C++*, *Oracle*, *Unix*, *program*}, an IR model often uses the query as a pattern to match relevant documents.
- However, the user profile may be some knowledge about a topic (representing as a set of terms and/or a term-weight distribution).
  - For example, we may treat "program" as topics (subjects) "programming language" or "programmer".
  - With this in mind, we can't just think of "program" as a pattern, but want to describe what we know about the topics of "programming language" or "programmer" to determine the relevant documents.

2

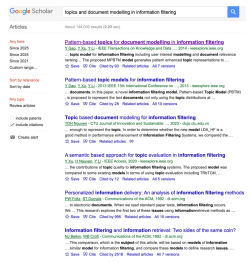
Key Differences between IF and IR

Aspect	IF	IR
Input	User information needs, profils or preferences	User query
Focus	Users	Documents
Interaction	Proactive	Reactive
Output	Stream of ranked relevant documents or relevant items	A ranked list of relevant dopcuments
Example	Spam filter, recommender system	Google search

3

User Information Needs

- *User information needs* refer to what users want to fulfill a particular goal or need.
- *User feedback* is a way to learn a user information need, which is the information provided by a user about their judgement or experience with a service, produce or feature, which can be used to improve or better understand user information needs.
  - E.g., the clicked article (the first article in the list of articles on the left) is very likely to be a relevant document for the query of "topics and document modelling in information filtering".
- If something is relevant for a person in relation to a given query or topic, we might say that the person needs the information for that query of topic.
  - For example, an *information need* is the underlying cause of the query that a person submits to a search engine.
  - We can describe it as *the motivation* for a person to search for information on the Web.
- The example on the right is an example designed by Google to support scholars in searching for what (relevant articles) a user needs for a research topic, such as topic and document modeling in information filtering.



4

Information needs issues

- However, sometimes it is difficult for people to define exactly what their information need is since that information is a gap in their knowledge.
  - E.g., a short query can only describe partial information need.
  - A short query can also be extended to a long query (or a topic, see the example on the right) by providing more details about the query, such as <description> and *<narrative> tags on the right topic-example*.
- We can use variety of dimensions to categorize or understand information needs, e.g.,
  - Number of relevant documents being sought
  - Type of information that is needed
  - Type of task that led to the requirement for information

Topic Example:

<topic>
<title> Economic espionage </title>
<description> What is being done to counter economic espionage internationally?
</description>
<narrative> Documents which identify economic espionage cases and provide action(s) taken to reprimand offenders or terminate their behaviour are relevant. Economic espionage would encompass commercial, technical, industrial or corporate types of espionage. Documents about military or political espionage would be irrelevant.
</narrative>
</topic>

5

2. Query-Based Methods for Capturing Information Needs

- A query can represent very different information needs
  - May require different search techniques and ranking algorithms to produce the best rankings
- A query can be a poor representation of the information need
  - Users may find it difficult to express their information needs as
    - users are encouraged to enter short queries both by the search engine interface, and by the fact that long queries don't work
- When a system uses user queries to acquire user information needs, the important part of query processing to correct **spelling errors** since there are about 10-15% of all web queries have spelling errors.
- Basic approach for checking spelling errors:
  - suggest corrections for words not found in *spelling dictionary*
  - Suggestions found by comparing word to words in dictionary using similarity measure
- Most common similarity measure is ***edit distance***
  - number of operations required to transform one word into the other

6

Edit Distance

- *Damerau-Levenshtein* distance
  - counts the minimum number of insertions, deletions, substitutions, or transpositions of single characters required
  - e.g., Damerau-Levenshtein distance 1

    extenssions → extensions(insertion error)
    poiner → pointer (deletion error)
    marshmellow → marshmallow (substitution error)
    brimingham → birmingham (transposition error)

  - distance 2 (doceration -> decoration)

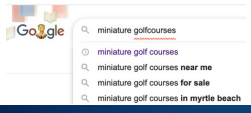
    doceration → deceration
    deceration → decoration
- Number of techniques used to speed up calculation of edit distances
  - restrict to words starting with same character
  - restrict to words of same or similar length
  - restrict to words that sound the same

7

Spellcheck Process and Query Suggestion

1. Tokenize the query.
2. For each token, a set of alternative words and pairs of words is found using an edit distance modified by weighting certain types of errors as described above. The data structure that is searched for the alternatives contains words and pairs from both the query log and the trusted dictionary.
3. The noisy channel model is then used to select the best correction.
4. The process of looking for alternatives and finding the best correction is repeated until no better correction is found.

e.g.,
miniture golfcurses
miniature golfcourses
miniature golf courses



8

Noisy Channel for Spelling Correction

- It is a general framework that can address the issues of ranking, context, and run-on errors.
- The user chooses word *w* based on a probability distribution $P(w)$
  - called the *language model*
  - can capture context information, e.g., $P(w_1|w_2)$
- The user tries to write word w, but the noisy channel causes the person to write word *e* instead, with a probability $P(e|w)$
  - called *error model*
  - represents information about the frequency of spelling errors; e.g., the probabilities for words (*e*) that are edit distance 1 away from, the word *w* will be quite high.

9

Noisy Channel Model

- Now the problem is that
  - The corrector (algorithm) observes an error *e* (golf ***curse***) and tries to suggest a correct work *w* (***course***), i.e.,
  - the probability that the correct word is *w* given that we can see the person wrote *e*.
- We need to estimate probability of correction
  - $P(w|e) = P(e|w)P(w)$
  - It is the product of the error model probability and the language model probability.
- Example:
  - Assume a user input, a query "fish tink"
  - The error *e* ="tink"
  - The correct words are possible "tank" or "think".
  - Which word we are going to recommend as the correct word?

10

Query Log Example

Timestamp	User ID	Query
2026-03-25 10:01	U001	fish tank setup
2026-03-25 10:03	U002	how to think clearly
2026-03-25 10:05	U001	tropical fish types
2026-03-25 10:07	U003	fish tank filter
2026-03-25 10:10	U002	think tank meaning
2026-03-25 10:12	U004	think best fish food
2026-03-25 10:15	U003	clean fish tank
2026-03-25 10:18	U002	think examples

- We need a query log to estimate the language model probability

11

Noisy Channel Model: Example

12

- We firstly estimate the **language model probability** by using context in a query log or a collection:
  $P(w) = \lambda p(w) + (1-\lambda)p(w|w_p)$, *where* w_p is the previous word of w.
- Let w = 'tank', and w_p = 'fish'. Then we have
  $P(w) = P(\text{'tank'}) = \lambda p(w) + (1-\lambda)p(w|w_p) = \lambda p(\text{'}tank\text{'}) + (1-\lambda)\, p(\text{'tank'} \mid \text{'fish'})$
- Let w = 'think', and w_p = 'fish'. Then we have
  $P(w) = P(\text{'think'}) = \lambda p(w) + (1-\lambda)p(w|w_p) = \lambda p(\text{'}think\text{'}) + (1-\lambda)\, p(\text{'think'} \mid \text{'fish'})$
- Based on the query log, we can infer that $p(\text{'}tank\text{'}) = p(\text{'}think\text{'})$, and $p(\text{'tank'} \mid \text{'fish'}) > p(\text{'think'} \mid \text{'fish'})$
- Then $P(\text{'tank'}) > P(\text{'think'})$

How to calculate words 'tank's and 'think's probabilities in the query log? → $P(q_i|D) = \frac{f_{q_i,D}}{|D|}$

How to calculate conditional probabilities in the query log? → $P(A \mid B) = \frac{P(A \cap B)}{P(B)}$

12

Query Log Example cont.

13

Timestamp	User ID	Query
2026-03-25 10:01	U001	fish tank setup
2026-03-25 10:03	U002	how to think clearly
2026-03-25 10:05	U001	tropical fish types
2026-03-25 10:07	U003	fish tank filter
2026-03-25 10:10	U002	think tank meaning
2026-03-25 10:12	U004	think best fish food
2026-03-25 10:15	U003	clean fish tank
2026-03-25 10:18	U002	think examples

- $p(\text{'}tank\text{'}) = p(\text{'}think\text{'}) = 4 / 25$
- $p(\text{'tank'} \mid \text{'fish'}) = 3 / 5$
- $p(\text{'think'} \mid \text{'fish'}) = 1 / 5$

13

Noisy Channel Model cont.

14

- Estimating error model probability $P(e|w)$
  - The simple approach: assume all words with same edit distance have same probability, only edit distance 1 and 2 considered.
  - More complex AI-based approach: incorporate estimates based on common typing errors.
    - These estimates are derived from large collections of text by finding many pairs of correctly and incorrectly spelled words.
- Based on the above simple approach, $P(\text{'tink'} \mid \text{'tank'}) = P(\text{'tink'} \mid \text{'think'})$
- Thus, since $P(\text{'tank'} \mid \text{'tink'}) > P(\text{'think'} \mid \text{'tink'})$, and we infer that the most likely correction is "tank".

14

3. Relevance Feedback Using Machine Learning

15

- **Relevance feedback** is a popular technique to support machine learning algorithms to learn information needs or user profiles by indicating which documents are interesting (i.e., relevant) and possibly which documents are completely off-topic (i.e., non-relevant).
- **Explicit Feedback**
  - The user directly indicates which documents are relevant or irrelevant.
  - How it works
    - After reviewing the initial search results, the user marks documents as either "relevant" or "not relevant." The system then uses this feedback to learn the user's relevance criteria and better align future search results with the user's information needs.
  - Example:
    - In a library search system, the user can click "relevant" on a set of returned books.
  - Pros: Highly accurate; tailored to user's actual preferences.
  - Cons: Requires extra effort from the user.

15

Implicit Feedback

16

- Systems infer relevance from user behavior rather than direct input.
- How it works
  - The system tracks actions such as clicks, dwell time, scrolling, downloads, or page visits to estimate which documents are relevant.
- Example:
  - A user clicks on three articles about Python, ignores the others. Then the system assumes clicked articles are relevant.
  - A machine learning–based system can then refine queries or identify additional relevant terms from the documents marked as "relevant." For example,
    - it may select terms that occur more frequently in relevant documents than in non-relevant ones, or automatically reformulate a query by adding new terms and adjusting the weights of the original terms.
    - A revised ranking of documents can then be generated based on this modified query.
- Pros: No extra effort from the user; scalable.
- Cons: Less precise; can misinterpret user behavior.

16

Pseudo-relevance feedback

17

- Systems assume the top-ranked documents from an initial query-based search are relevant without asking the user.
- How it works:
  - Perform an initial query → retrieve top-k documents.
  - Assume these documents are relevant → expand or refine the query as the user information needs.
  - Re-run the query with the enhanced query.
- Example:
  - Query: "machine learning for NLP" → top 5 results are used to add new terms like "neural network", "BM25", "tf*idf".
- Pros: Fully automatic; improves results without user input.
- Cons: May reinforce incorrect assumptions if top documents are not truly relevant (or the precision is not very high).

17

Uncertainty factors in pseudo-relevance feedback

18

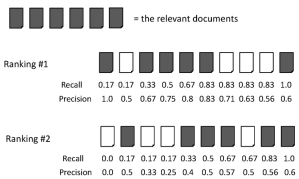
- Pseudo relevance feedback inherently involves uncertainties. If a query contains the same terms as another query, the retrieved results will be identical, regardless of:
  - who submitted the query,
  - why the query was submitted,
  - where the query was submitted, or
  - what other queries were submitted in the same session.
- In this approach, the only factor that influences the performance of systems is the specific words used to formulate the information needs.
- However, other factors, collectively known as **query context** (e.g., user profiles, privacy considerations, location), can significantly affect the perceived relevance of the results.

18

4. Ranking Effectiveness

19

- Most of IR or IF models we have discussed produce ranked output.
- We can use recall, precision and F measure to evaluate the ranking results; however, rank positions also make sense as people usually want to put the very relevant information at the top of the output.
- In the example on the right-hand side, Ranking #1 and Ranking #2 achieve the same precision and recall when retrieving 10 documents.
- However, Ranking #1 appears to be superior in terms of the ordering of relevant documents.
- **How to compare ranking #1 (model A) and ranking #2 (model B) for the same query?**



19

Summarizing a Ranking

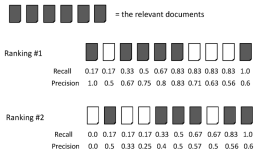
20

- There are three popular methods to measure ranking effectiveness:
  - Averaging the precision values from the rank positions where a relevant document was retrieved.
    - It is a single number that is based on the ranking of all the relevant documents, but the value depends heavily on the highly ranked relevant documents.
  - Calculating recall and precision **at fixed** rank positions (or cutoff)
    - For a given query, to compare two or more rankings, the precision at the predefined rank positions needs to be calculated. This effectiveness measure is known as *precision at rank p*.
  - Calculating precision at standard recall levels, from 0.0 to 1.0 in increments of 0.1
    - Each ranking is then represented using 11 numbers. This idea can be extended to calculate averaging effectiveness across queries and generating recall-precision graphs (*interpolation*).

20

Example of Average Precision

21



Average precision summarizes the ranking for a given query by averaging the precision values at the positions where relevant documents are retrieved.

The average precision of Ranking #1 is 0.78, while that of Ranking #2 is 0.52; therefore, Ranking #1 outperforms Ranking #2.

$$\text{Ranking } \#1\text{: } (1.0 + 0.67 + 0.75 + 0.8 + 0.83 + 0.6)/6 = 0.78$$

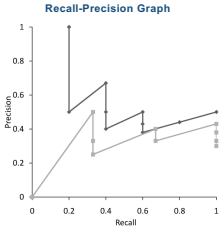
$$\text{Ranking } \#2\text{: } (0.5 + 0.4 + 0.5 + 0.57 + 0.56 + 0.6)/6 = 0.52$$

21

Averaging Across Queries

22

- *Mean Average Precision* (**MAP**)
  - summarize rankings from multiple queries by averaging average precision
  - most commonly used measure in research papers
  - assumes user is interested in finding many relevant documents for each query
  - requires many relevance judgments in text collection
- Recall-precision graphs are also useful summaries, which give more detail on the effectiveness of the ranking algorithm at different recall levels.

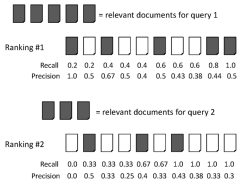


22

MAP Example

23

Consider an IR model that produces two rankings, Ranking #1 and Ranking #2, corresponding to two queries, query 1 and query 2.



$$average\ precision\ query\ 1 = (1.0 + 0.67 + 0.5 + 0.44 + 0.5)/5 = 0.62$$
$$average\ precision\ query\ 2 = (0.5 + 0.4 + 0.43)/3 = 0.44$$

$$mean\ average\ precision = (0.62 + 0.44)/2 = 0.53$$

23

Focusing on Top Documents

24

- Users tend to look at only the top part of the ranked result list to find relevant documents
- Some search tasks have only one relevant document
  - e.g., navigational search, question answering
- Recall not appropriate
  - instead need to measure how well the search engine does at retrieving relevant documents at very high ranks

- Precision at Rank R
  - R typically 5, 10, 20
  - easy to compute, average, understand
  - not sensitive to rank positions less than R
- Reciprocal Rank
  - reciprocal of the rank at which the first relevant document is retrieved
  - *Mean Reciprocal Rank (MRR)* is the average of the reciprocal ranks over a set of queries
  - very sensitive to rank position

24

5. Interpolation

25

- Precision are often calculated at discrete recall points. However, to get a smoother and more interpretable performance curve, interpolation is applied.
- An **interpolated precision at a standard recall level** R is defined as:

$$P(R) = \max\{P' : R' \geq R \wedge (R', P') \in S\}$$

where S is the set of observed (R, P) points.
The standard recall levels are 0.0 to 1.0 in increments of 0.1.

- It defines precision at any standard recall level as the *maximum* precision observed in any recall-precision point at a higher recall level
  - produces a step function
  - can define precision at recall 0.0

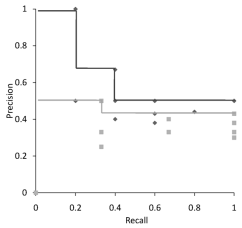
25

Interpolation Precision Example

26

- Explanation of the equation $P(R)$
  - The equation can be understood in two steps:
    - 1) create a set via set comprehension, and
    - 2) take the maximum value from the set.
- Example, for Ranking #1, we have
  S = {(0.2, 1.0), (0.2, 0.5), (0.4, 0.67), (0.4, 0.5), (0.4, 0.4), (0.6, **0.5**), (0.6, **0.43**), (0.6, **0.38**), (0.8, **0.44**), (1.0, **0.5**)},
  1) If R = 0.6, the set = {0.5, 0.43, 0.38, 0.44, 0.5}.
  2) max{0.5, 0.43, 0.38, 0.44, 0.5} = 0.5

  Therfore, $P(0.6) = 0.5$
- The figure on the right shows the interpolated curves for both Rankings #1 and #2.



26

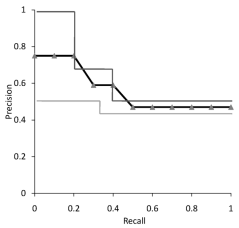
Average Recall-Precision Graph

27

Average Precision at Standard Recall Levels

Recall	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
Ranking 1	1.0	1.0	1.0	0.67	0.67	0.5	0.5	0.5	0.5	0.5	0.5
Ranking 2	0.5	0.5	0.5	0.5	0.43	0.43	0.43	0.43	0.43	0.43	0.43
Average	0.75	0.75	0.75	0.59	0.47	0.47	0.47	0.47	0.47	0.47	0.47

Recall-precision graph plotted by simply joining the average precision points at the standard recall levels



27

6. Discounted Cumulative Gain

28

- Popular measure for evaluating web search and related tasks
- Two assumptions:
  - Highly relevant documents are more useful than marginally relevant document
  - the lower the ranked position of a relevant document, the less useful it is for the user, since it is less likely to be examined.
- Uses *graded relevance* as a measure of the usefulness, or *gain*, from examining a document.
- Gain is accumulated starting at the top of the ranking and may be reduced, or *discounted*, at lower ranks.
- Typical discount is 1/*log (rank)*
  - With base 2, the discount at rank 4 is 1/2, and at rank 8 it is 1/3

28

Discounted Cumulative Gain: *DCG* function

29

- *DCG* is the total gain accumulated at a particular rank *p*:
- Alternative formulation:

$$DCG_p = rel_1 + \sum_{i=2}^{p} \frac{rel_i}{\log_2 i}$$

$$DCG_p = \sum_{i=1}^{p} \frac{2^{rel_i} - 1}{log(1+i)}$$

  - Where rel_i is the graded relevance level of the document retrieved at rank *i*.
    - E.g., Relevance Judgments on a six-point scale will be ranged from "Bad" to "Perfect" (i.e., 0 ≤ rel_i ≤ 5).
    - Binary relevance judgments can also be used, in which case rel_i would be either 0 or 1.
  - DCG is used by some web search companies
  - Its emphasis on retrieving highly relevant documents

29

30

DCG Example

- 10 ranked documents judged on 0-3 relevance scale:
  3, 2, 3, 0, 0, 1, 2, 2, 3, 0
- Discounted gain:
  3, 2/1, 3/1.59, 0, 0, 1/2.59, 2/2.81, 2/3, 3/3.17, 0
  = 3, 2, 1.89, 0, 0, 0.39, 0.71, 0.67, 0.95, 0
- DCG at each rank:
  3, 5, 6.89, 6.89, 6.89, 7.28, 7.99, 8.66, 9.61, 9.61

30

31

Normalized DCG

- DCG numbers are averaged across a set of queries at specific rank values
  - e.g., DCG at rank 5 is 6.89 and at rank 10 is 9.61
- DCG values are often *normalized* by comparing the DCG at each rank with the DCG value for the *perfect ranking*
  - makes averaging easier for queries with different numbers of relevant documents.

NDCG **Example**

- Perfect ranking:
  3, 3, 3, 2, 2, 2, 1, 0, 0, 0
- ideal DCG values:
  3, 6, 7.89, 8.89, 9.75, 10.52, 10.88, 10.88, 10.88, 10.88
- NDCG values (divide actual by ideal):
  1, 0.83, 0.87, 0.76, 0.71, 0.69, 0.73, 0.8, 0.88, 0.88
  - NDCG ≤ 1 at any rank position.

31

32

Using Preferences

- Two rankings described using preferences can be compared using the *Kendall tau coefficient (τ):*

$$\tau = \frac{P - Q}{P + Q}$$

  - *P* is the number of preferences that agree, and *Q* is the number that disagree
- For preferences derived from binary relevance judgments, we can use *BPREF*.

32

33

BPREF

- For a query with *R* relevant documents, only the first *R* non-relevant documents are considered

$$BPREF = \frac{1}{R}\sum_{d_r}(1 - \frac{N_{d_r}}{R})$$

  - d_r is a relevant document, and N_{dr} gives the number of non-relevant documents that are ranked higher than d_r.
- Alternative definition

$$BPREF = \frac{P}{P+Q}$$

33

34

References

[1] Ted Kwartler, *Text Mining in Practice with R*, Wiley, 2017.
[2] Chapter 6 in book - W. Bruce Croft, *Search Engines - Information retrieval in Practice*; Pearson, 2010.
[3] Y. Li, A. Algarni, M. Albathan, Y. Shen and M. Bijaksana, Relevance Feature Discovery for Text Mining, *IEEE Transactions on Knowledge and Data Engineering*, 2015, vol. 27, no. 6, pages 1656-1669.
[4] D. Liu, et al., RecPrompt: A Self-tuning Prompting Framework for News Recommendation Using Large Language Models, in CIKM '24: Proceedings of the 33rd ACM International Conference on Information and Knowledge Management, Pages 3902 – 3906, 2024, https://doi.org/10.1145/3627673.3679987
[5] Kim Martineau, What is retrieval-augmented generation? https://research.ibm.com/blog/retrieval-augmented-generation-RAG

34